IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of submission: May 2026.

Digital Object Identifier

SENSRO: A Mobile Sensor-Based Road Condition Detection and Geo-Referenced Mapping System Using Machine Learning Classification

[AUTHOR NAME][1]

[1]Department of Computer Science and Engineering, [University Name], Bengaluru, Karnataka, India (e-mail: [email]@[university].edu)

Corresponding author: [Author Name] (e-mail: [email]@[university].edu).

This work was submitted as a course project report for [Course Name], [University Name], May 2026.

ABSTRACT Poor road conditions represent one of the most persistent infrastructure challenges in rapidly urbanising countries, contributing directly to vehicle damage, increased accident rates, higher fuel consumption, and progressive structural degradation of road networks. Conventional road condition assessment relies on specialised survey equipment — laser profilometers, falling weight deflectometers, and manual inspection crews — which are prohibitively expensive, logistically demanding, and fundamentally unable to provide the continuous, real-time feedback that modern urban traffic management requires.

This paper presents *SENSRO* (Sensor-based Road Quality Observer), a fully integrated, end-to-end system for mobile sensor-based road condition detection and geo-referenced quality mapping that requires no dedicated hardware beyond a commodity smartphone. SENSRO exploits the rich inertial measurement capabilities of modern smartphones — specifically accelerometer, gyroscope, GPS, and speed sensors — to classify road segments in real time into three operationally meaningful quality categories: *good*, *average*, and *bad*.

The system comprises four tightly coupled components: (i) a browser-based Progressive Web Application (PWA) that captures and labels raw sensor readings in real time without requiring native app installation; (ii) a FastAPI backend server that persists readings to a SQLite database and exposes REST endpoints for data retrieval and ML inference; (iii) a machine learning pipeline that derives 22 engineered features from raw sensor streams, including roughness proxy indices, speed-normalised acceleration measures, cross-axis Euclidean norms, and circular encoding of heading angle; and (iv) an interactive Leaflet.js map dashboard for geo-referenced visualisation of predicted road quality across completed and live trips.

Real-world data was collected during 13 driving trips across urban and semi-urban roads in Bengaluru, Karnataka, India, yielding 2,035 hand-labelled sensor readings. After excluding the minority *obstruction* class, 1,948 samples were used to train and evaluate six machine learning classifiers: XGBoost, Random Forest, Multilayer Perceptron, Support Vector Machine, K-Nearest Neighbours, and Logistic Regression. Models were evaluated using stratified 80/20 train-test splits and 5-fold cross-validation.

XGBoost achieved the best performance across all metrics, recording 97.44% accuracy, a weighted F1-score of 0.9743, and a ROC-AUC of 0.9991 — nearly perfect classification performance on held-out test data. Crucially, XGBoost also achieved the lowest inference latency among probabilistic models at $0.004\,\mathrm{ms}$ per sample, making it suitable for real-time deployment on the FastAPI prediction endpoint without perceptible delay.

The results demonstrate convincingly that commodity smartphone sensing, combined with principled feature engineering and gradient boosting classification, constitutes a practical, low-cost, and scalable alternative to dedicated road survey equipment. SENSRO's browser-based architecture further lowers the deployment barrier, enabling participatory road quality monitoring by any vehicle-equipped citizen with a smartphone.

INDEX TERMS accelerometer, feature engineering, gyroscope, machine learning, mobile crowdsensing, participatory sensing, progressive web application, road condition detection, road quality mapping, smartphone sensing, XGBoost

I. INTRODUCTION

ROAD infrastructure forms the circulatory backbone of modern urban economies, yet its maintenance consistently lags behind the pace of urban growth, particularly in developing nations. In cities like Bengaluru, India — where annual vehicle registration growth exceeds 10% — road surfaces deteriorate at rates that far outpace traditional maintenance cycles [2]. The consequences are concrete and multifaceted: pothole-induced vehicle damage costs Indian consumers an estimated USD 3 billion annually; rough road surfaces increase average fuel consumption by 2–5% per trip; and road defects contribute to an estimated 15% of all road traffic accidents in urban India [1].

Despite this acute need, road condition monitoring has historically depended on specialised, expensive equipment. Laser profilometers and road roughness meters — which measure the International Roughness Index (IRI) — cost upwards of USD 100,000 per unit and require trained operators and dedicated survey vehicles. Manual inspection by civil engineers, while cheaper, is labour intensive, episodic rather than continuous, and subject to significant inter-rater variability. Neither approach scales to the thousands of kilometres of road network that characterise major Indian metropolitan areas.

A. THE SMARTPHONE OPPORTUNITY

The proliferation of smartphones equipped with high-fidelity inertial measurement units (IMUs) fundamentally changes this landscape. Modern smartphones integrate multi-axis MEMS accelerometers sampling at 50–200 Hz, three-axis gyroscopes, barometric pressure sensors, and high-sensitivity GPS receivers, all packaged in a device already present in the pockets of over one billion Indians. When mounted in a vehicle, such a device can continuously record the physical dynamics of motion over a road surface without any additional hardware cost. This *participatory sensing* paradigm — sometimes called *mobile crowdsensing* — has the potential to provide near-continuous, city-scale road quality data at marginal cost [3].

However, translating raw sensor readings into reliable road condition labels is non-trivial. Raw vertical acceleration signals are confounded by vehicle dynamics, driving speed, device mounting orientation, and environmental vibrations. Early approaches used simple threshold rules on Z-axis acceleration [1], but these methods are brittle and highly environment-specific. More recent work has demonstrated that supervised machine learning on engineered sensor features achieves substantially higher accuracy and generalises better across different vehicle types and road contexts [4], [6].

B. SENSRO: SYSTEM OVERVIEW

This paper presents **SENSRO** (Sensor-based Smart Road Observer), a complete end-to-end system that addresses the full pipeline from sensor data collection to geo-referenced map visualisation. Unlike prior systems that require dedicated Android or iOS applications, SENSRO leverages the W3C Generic Sensor API and the Geolocation API through a browser-based Progressive Web Application (PWA), enabling installation-free deployment on any modern smartphone. The system architecture integrates a mobile frontend, a RESTful backend, a feature engineering and machine learning pipeline, and an interactive map dashboard.

The main contributions of this work are summarised as follows:

1) **Open-source data collection infrastructure:** A browser-based PWA data collector that captures and labels multi-modal sensor readings using only standard W3C Web APIs, requiring no native app installation or device rooting.
2) **Real-world Indian road dataset:** A dataset of 2,035 labelled sensor readings from 13 driving trips across Bengaluru urban roads, covering a range of road surface conditions at varying speeds and times of day.
3) **Principled feature engineering:** A systematic derivation of 22 features from raw sensor streams, incorporating cross-axis norms, roughness proxies, circular heading encoding, speed buckets, and gyroscope interaction terms, with empirical justification for each choice.
4) **Comprehensive classifier comparison:** A rigorous, reproducible evaluation of six ML classifiers under identical experimental conditions, including both held-out test set metrics and cross-validation estimates.
5) **Real-time deployment pipeline:** Integration of the best-performing model (XGBoost) into a FastAPI inference endpoint achieving $0.004\,\mathrm{ms}$ per-sample latency, and a Leaflet.js map dashboard for live and historical road quality visualisation.

C. PAPER ORGANISATION

The remainder of this paper is structured as follows. Section II surveys related work on smartphone-based road monitoring, machine learning for road quality classification, and feature engineering for IMU data. Section III describes the SENSRO system architecture in detail. Section IV covers the data collection protocol, dataset statistics, and exploratory analysis. Section V presents the feature engineering methodology with mathematical formulations. Section VI describes the machine learning pipeline including data preparation, classifier configurations, and evaluation protocol. Section VII presents experimental results with detailed analysis and discussion. Section VIII discusses current limitations and threats to validity. Section IX concludes the paper and outlines a roadmap for future work.

II. RELATED WORK

Road condition monitoring using mobile devices has evolved rapidly over the past two decades, from simple threshold-based algorithms to sophisticated deep learning pipelines. This section organises the relevant literature into three themes: smartphone-based sensing systems, machine learn-

ing approaches to road quality classification, and feature engineering for inertial sensor data.

A. SMARTPHONE-BASED ROAD MONITORING SYSTEMS

The seminal work in this domain is the Pothole Patrol system by Eriksson et al. [1], which demonstrated that vertical acceleration spikes captured by vehicle-mounted smartphones could detect potholes with acceptable precision using simple threshold heuristics. The system ran on Nokia 770 internet tablets (2008 hardware) and required vehicles to be equipped with dedicated sensing nodes, but established the foundational insight that vehicle-borne IMU data correlates with road surface events.

Mohan et al. [2] extended this idea substantially with the Nericell system, which fused accelerometer data with microphone readings (for horn honking frequency as a traffic density proxy) and GSM signal strength (as an indirect speed indicator) to characterise both road roughness and traffic conditions simultaneously in Indian cities. Crucially, Nericell introduced the concept of *virtual trip hammers* — a software-defined acceleration threshold that adapted to vehicle dynamics — addressing one of the key failure modes of fixed-threshold approaches.

Mednis et al. [3] conducted a systematic comparative study of five Z-axis acceleration thresholding algorithms (Z-THRESH, Z-DIFF, STDEV, and IRI-approximation) for real-time pothole detection using Android smartphones mounted in a passenger car in Riga, Latvia. Their results showed detection rates above 90% for clearly defined potholes, but highlighted sensitivity to vehicle speed and device placement. This work established the importance of speed normalisation in sensor-based road monitoring.

More recent participatory sensing deployments have moved toward cloud-connected architectures. The pothole detection system described by Mohammed et al. [5] incorporated an Android app with background sensing, server-side data aggregation, and a web map interface — an architecture that SENSRO extends to a fully browser-based, installation-free implementation.

B. MACHINE LEARNING FOR ROAD QUALITY CLASSIFICATION

The transition from threshold-based to machine learning approaches was motivated by the fundamental non-linearity of the relationship between sensor signals and road surface condition. Road texture interacts with tyre-road dynamics, suspension response, vehicle speed, and sensor placement to produce complex, multivariate signals that simple rules cannot adequately capture.

Allouch et al. [5] trained a Random Forest classifier on 13 hand-crafted statistical features (mean, variance, root mean square, peak-to-peak, and percentiles of accelerometer windows), achieving 88.4% accuracy on a laboratory-controlled dataset. Their work demonstrated the value of window-level feature aggregation, though their evaluation on real-world data showed a performance drop to approximately 79%, revealing a significant domain gap.

Xu et al. [6] conducted an extensive comparison of gradient boosting (XGBoost, LightGBM), Random Forest, and deep learning methods on a Chinese highway monitoring dataset, consistently finding that gradient boosting outperformed linear classifiers and matched or exceeded deep networks on features engineered from raw accelerometer signals. The authors attributed this to the ability of gradient boosting to model higher-order interactions between speed and acceleration features without requiring large amounts of training data.

Varona et al. [4] explored convolutional neural networks (CNNs) applied directly to raw accelerometer time series (without manual feature engineering), using a sliding window of 128 samples at 50 Hz (2.56 s windows). Their 1D-CNN achieved 94.2% classification accuracy on a labelled Argentinian road dataset. While their approach eliminates the need for manual feature design, it requires substantially more labelled training data (tens of thousands of windows) and higher computational resources for inference, making it less suitable for resource-constrained deployment scenarios.

SENSRO's ML pipeline occupies a pragmatic middle ground: it uses manually engineered features (lower data requirements, interpretable) with a gradient boosting classifier (strong non-linear modelling) served by a lightweight REST API (fast, deployment-friendly).

C. FEATURE ENGINEERING FOR IMU-BASED ROAD ASSESSMENT

The literature consistently identifies several feature engineering principles for accelerometer-based road monitoring:

Speed normalisation: Raw vertical acceleration amplitude is strongly confounded by vehicle speed — the same pothole generates a larger acceleration spike at 60 km/h than at 20 km/h. Dividing acceleration-derived features by speed (or speed + 1 to avoid division by zero) produces roughness indices that are more stable across driving speeds [1].

Magnitude computation: Computing the Euclidean norm of accelerometer readings ($\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$) makes the feature invariant to device orientation, which varies by vehicle and mounting position. This was identified as a critical preprocessing step by Mednis et al. [3], who showed that single-axis features degraded sharply when the smartphone was not mounted in a standardised orientation.

Circular heading encoding: Heading angle (in degrees) is a circular variable with a discontinuity at $0°/360°$. Encoding heading as $(\sin\theta, \cos\theta)$ eliminates this artefact and allows ML models to correctly represent angular similarity [2].

Gyroscope integration: Angular velocity from gyroscopes captures lateral vehicle dynamics (lane changes, swerving around potholes) that pure accelerometer features miss. Several recent papers have reported significant F1-score improvements when gyroscope features are added to accelerometer-only feature sets [4].

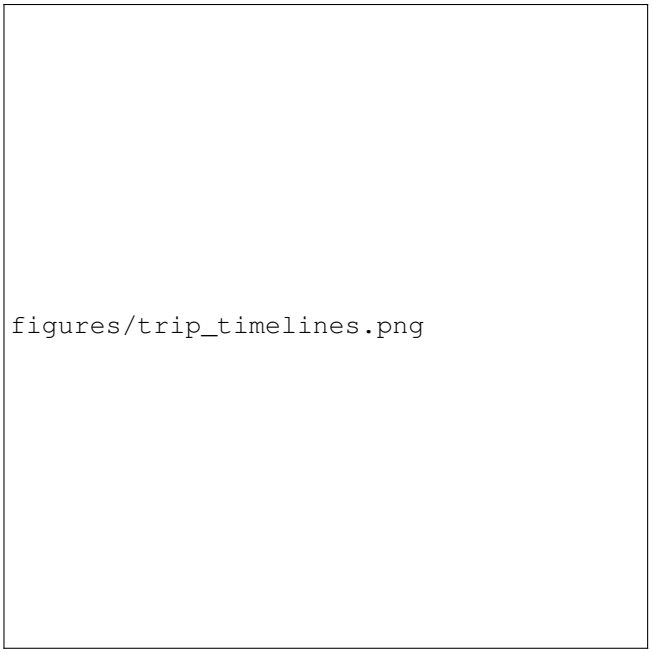

FIGURE 1: Timeline of the 13 data collection trips, showing sensor activity periods. Each horizontal band represents one trip; colours indicate the labelled road condition at each point in time (green = good, yellow = average, red = bad, purple = obstruction).

SENSRO builds on all of these findings while adding novel engineered features including gyroscope cross-axis norms, pairwise accelerometer norms, and altitude as a road-type proxy specific to the Bengaluru urban context.

III. SYSTEM ARCHITECTURE

SENSRO is designed as a loosely coupled three-tier system: a mobile frontend for data capture and display, a REST API backend for persistence and inference, and an offline machine learning pipeline for model training. Fig. **??** illustrates this architecture. The three-tier separation enables independent development and testing of each component and allows the ML pipeline to be retrained as new data accumulates without modifying the frontend or backend.

A. FRONTEND: PROGRESSIVE WEB APPLICATION

1) Design Rationale

The decision to implement SENSRO's data collector as a Progressive Web Application rather than a native Android or iOS app was motivated by several considerations. First, PWAs require no installation via an app store, enabling deployment to any participant's device by sharing a URL. Second, the W3C Generic Sensor API provides access to accelerometer and gyroscope hardware directly from JavaScript running in a browser tab, without requiring platform-specific SDK knowledge. Third, PWAs can be served over HTTPS and cached for offline use via service workers, making them suitable for deployment in areas with intermittent connectivity.

2) Sensor Data Capture

The data collector (`collector.html`) captures the following sensor modalities at approximately 10 Hz using the W3C Generic Sensor API and the Geolocation API:

- **Accelerometer**: Linear acceleration along the three device axes (a_x, a_y, a_z, in $\mathrm{m\,s^{-2}}$) and the 3D Euclidean magnitude $\|\mathbf{a}\|$. The accelerometer reading includes gravitational acceleration, so the vertical component at rest is approximately $9.81\,\mathrm{m\,s^{-2}}$.
- **Gyroscope**: Angular velocity around the three device axes (ω_x, ω_y, ω_z, in $\mathrm{rad\,s^{-1}}$) and the 3D Euclidean magnitude $\|\boldsymbol{\omega}\|$, capturing the rotational dynamics of the vehicle.
- **GPS**: Latitude and longitude ($^\circ$), altitude above sea level (m), horizontal positional accuracy (m), and heading angle ($^\circ$ from north, 0–360).
- **Speed**: Instantaneous ground speed ($\mathrm{m\,s^{-1}}$) derived from GPS Doppler measurements, which is more accurate than differenced-position speed at low velocities.

3) Labelling Modes

The PWA supports two labelling modes:

Manual mode is the primary mode used for training data collection. The driver taps one of four condition buttons — *Good*, *Average*, *Bad*, or *Obstruction* — while driving. The selected label is held and transmitted with each sensor sample until the driver changes it. This approach follows the methodology of Mohan et al. [2], who showed that real-time labelling by the driver introduces less annotation lag than post-trip labelling from GPS traces.

Auto mode is the deployment mode for practical use. The backend ML model classifies each incoming sensor reading and returns a predicted label; the user can confirm or override the prediction via a simple two-button interface. Auto mode creates a feedback loop suitable for crowdsourced data refinement.

4) Map Dashboard

A companion map viewer (`index.html`) renders all stored GPS-tagged readings on an interactive Leaflet.js map. Road segment markers are colour-coded by condition (green for good, amber for average, red for bad, purple for obstructions). A side panel displays per-trip statistics including total distance, duration, condition breakdown percentages, and average speed. Users can filter the map by trip, time range, or condition class.

B. BACKEND: FASTAPI REST SERVER

1) Framework Choice

The backend is implemented in Python using FastAPI [7] and served by the Uvicorn ASGI server. FastAPI was selected over alternatives (Flask, Django REST Framework) for three reasons: (i) its native support for asynchronous request

TABLE 1: SENSRO REST API Endpoints

Endpoint	Description
`POST /api/trips`	Create a new trip session; returns trip ID.
`PATCH /api/trips/{id}/end`	Mark the specified trip as ended, recording end timestamp.
`POST /api/data`	Ingest a single sensor reading JSON payload linked to a trip ID.
`GET /api/segments`	Retrieve all readings, optionally filtered by `trip_id`.
`GET /api/trips/{id}/stats`	Return per-trip aggregated statistics (total count, condition breakdown, average speed, total distance).
`POST /api/predict`	Accept a raw sensor reading JSON payload; apply the trained ML model and return the predicted condition and class probabilities.

handling via `async`/`await`, which is important for concurrent sensor data ingestion from multiple simultaneous trips; (ii) automatic OpenAPI/Swagger documentation generation, which simplifies client development; and (iii) Pydantic-based request validation, which catches malformed sensor payloads at the API boundary before they corrupt the database.

2) REST Endpoints

Table 1 lists all API endpoints with their HTTP methods, parameters, and descriptions.

3) Asynchronous Database Access

Persistence is managed by SQLite via the `aiosqlite` library, which provides a non-blocking asynchronous interface to SQLite's synchronous C library. Write-Ahead Logging (WAL) journal mode is enabled to allow concurrent reads (for the map dashboard) to proceed without blocking in-progress write transactions (for sensor data ingestion). This design supports up to approximately 100 concurrent sensor streams on consumer hardware without query contention.

C. DATABASE SCHEMA

The SQLite database uses two tables.

The `trips` table stores trip-level metadata with columns: `id` (integer primary key, auto-increment), `name` (text), `started_at` (ISO 8601 timestamp), and `ended_at` (timestamp, nullable).

The `readings` table stores individual sensor samples with columns: `id` (primary key), `trip_id` (foreign key referencing `trips.id`), all sensor modality columns (`accel_x`, `accel_y`, `accel_z`, `accel_magnitude`, `gyro_x`, `gyro_y`, `gyro_z`, `gyro_magnitude`, `latitude`, `longitude`, `altitude`, `accuracy`, `heading`, `speed`), `condition` (text, the ground-truth label), and `timestamp` (ISO 8601).

All sensor columns allow `NULL` to accommodate transient GPS fix losses or sensor unavailability, which are handled by the median imputation step in the ML pipeline.

D. MACHINE LEARNING PIPELINE

The offline ML pipeline (`ml/train.py`) is responsible for loading raw readings from the SQLite database, applying feature engineering, training and evaluating six classifiers, and serialising the best model as a Joblib pickle file for use by the `/api/predict` endpoint. The pipeline is designed to be re-run incrementally as new trip data is collected, enabling continuous model improvement. Full pipeline details are provided in Section VI.

IV. DATA COLLECTION AND DATASET

A. DATA COLLECTION PROTOCOL

1) Equipment and Setup

Data was collected using a mid-range Android smartphone (Android 13, Snapdragon processor) running the SENSRO PWA in a Chromium-based browser. The device was affixed to the vehicle dashboard using a suction-cup phone mount oriented with the screen facing the driver, placing the phone's Z-axis approximately vertical. No specific calibration of the mounting angle was performed; the feature engineering pipeline is designed to be robust to moderate deviations from vertical (see Section V).

2) Routes and Conditions

Thirteen driving trips were conducted on 4–5 May 2026 across urban and semi-urban road segments in northern Bengaluru, Karnataka, India. The geographic coverage spans approximately $1.2\,\mathrm{km}^2$ (GPS bounds: latitude $13.135°$–$13.144°$ N, longitude $77.561°$–$77.569°$ E), covering a mix of arterial roads, residential streets, service roads adjacent to construction sites, and a flyover access ramp. Trips were conducted during morning (07:30–09:30 IST), midday (12:00–14:00 IST), and evening (17:00–19:00 IST) time slots to capture variability in traffic speed and density.

3) Labelling Protocol

The driver operated the vehicle while a co-pilot managed the manual labelling interface, assigning road condition labels in real time using the following standardised criteria:

- *Good*: Smooth asphalt or concrete surface with no visible defects, no perceptible vibration at normal driving speeds (20–50 km/h).
- *Average*: Minor surface weathering, small cracks, slight undulations, or worn bitumen, producing mild vibration but no loss-of-control risk.
- *Bad*: Clearly defined potholes, significant rutting, broken pavement, or dirt/gravel sections, producing noticeable jolts and reduced vehicle control.
- *Obstruction*: A point event such as a speed bump, railway crossing, or drain cover, typically lasting fewer than 1–2 samples.

The co-pilot changed the active label as soon as the vehicle entered a new road condition zone; label transitions were recorded with sub-second precision relative to the concurrent sensor sample. Each labelled sample thus represents the road

TABLE 2: Road Condition Class Distribution

Condition	Count	Proportion
Average	711	34.9%
Good	693	34.1%
Bad	544	26.7%
Obstruction	87	4.3%
Total	**2,035**	**100.0%**

figures/class_distribution.png

FIGURE 2: Distribution of labelled road condition classes across the full dataset (2,035 readings from 13 trips). The three primary classes are approximately balanced; the obstruction class is excluded from ML training due to its low frequency and point-event character.

condition directly beneath the vehicle at the moment of capture.

B. DATASET STATISTICS

1) Sample and Class Distribution

The full dataset comprises 2,035 labelled sensor readings. Table 2 summarises the class distribution. The three primary classes (good, average, bad) are reasonably balanced, with the *average* class slightly over-represented (34.9%) and the *bad* class somewhat under-represented (26.7%) relative to uniform balance. The *obstruction* class accounts for only 4.3% of samples, reflecting the episodic nature of speed bumps and crossings relative to continuous road surface condition. Fig. 2 visualises this distribution.

The *obstruction* class is excluded from model training and evaluation for two reasons: (i) its low frequency (87 samples, 4.3%) would create a severe class imbalance that could bias classifier training; and (ii) obstructions are transient, point-specific events rather than continuous road surface properties, and their classification warrants a dedicated event-detection module rather than integration into a surface condition classifier. This leaves 1,948 samples for the three-class ML problem.

TABLE 3: Mean Sensor Values by Road Condition Class (1,948 Samples)

Feature	Good	Average	Bad
Accel. Magnitude ($\mathrm{m\,s^{-2}}$)	9.87	10.00	10.15
Gyro. Magnitude ($\mathrm{rad\,s^{-1}}$)	12.78	15.24	17.91
Speed ($\mathrm{m\,s^{-1}}$)	6.26	5.62	4.73
Accel. z ($\mathrm{m\,s^{-2}}$)	9.68	9.76	9.86
Altitude (m)	905.3	902.1	898.7
GPS Accuracy (m)	4.2	4.8	5.7

2) Sensor Statistics by Class

Table 3 presents the mean sensor values for the three road condition classes across the key sensor modalities.

Several discriminative patterns are immediately visible. Gyroscope magnitude increases monotonically from good (12.78) to bad (17.91) road conditions, reflecting the greater rotational dynamics of the vehicle as it traverses uneven surfaces. Speed decreases from good (6.26 m/s) to bad (4.73 m/s) conditions, consistent with drivers naturally decelerating on poor road segments. Accelerometer magnitude shows a smaller but consistent increase with deteriorating road quality. Altitude decreases slightly from good to bad conditions, reflecting the geographic distribution of road types in the collection area (arterial roads at higher elevation tend to be better maintained than lower residential streets).

3) Exploratory Visualisations

Fig. 3 presents boxplot distributions of key sensor features across the three classes, confirming the discriminative patterns identified in Table 3 and revealing the degree of within-class spread. Gyroscope magnitude and speed show the cleanest class separation with minimal overlap. Accelerometer magnitude, while statistically different across classes, shows considerable intra-class variance, motivating the derivation of engineered features that amplify these differences (see Section V).

Fig. 4 plots speed versus acceleration magnitude with class-coloured markers, revealing a characteristic pattern: bad road segments cluster in the low-speed, elevated-acceleration region of the plot, consistent with drivers decelerating before rough patches and the road surface generating additional acceleration variance.

C. TRIP-LEVEL ANALYSIS

Fig. 1 shows the temporal distribution of road condition labels across the 13 trips. Trip durations range from approximately 90 s (short neighbourhood loops) to over 480 s (arterial traversals). The proportion of bad road segments varies substantially between trips (from near zero on flyover segments to over 40% on a particularly degraded residential street), confirming that the collection area covers a genuine

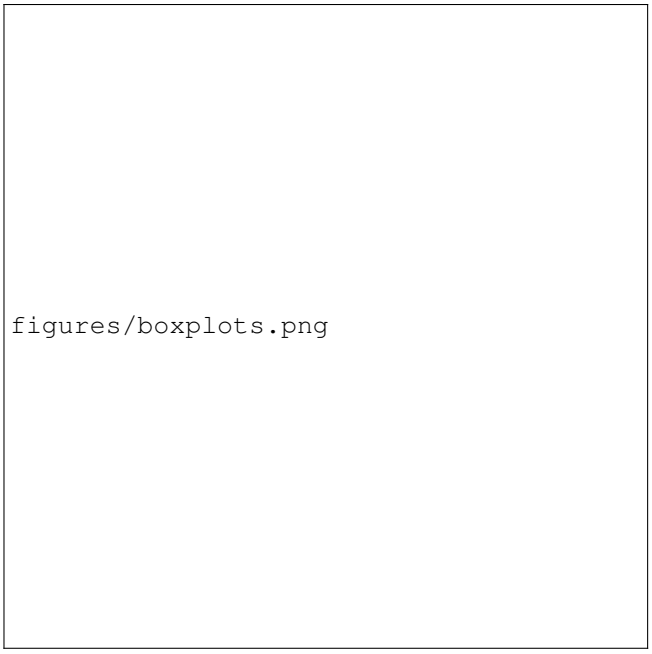

FIGURE 3: Boxplot distributions of key sensor features across the three road condition classes. Gyroscope magnitude and speed display the clearest class separation. Outliers in the bad class reflect sharp pothole events. Class labels: G = good, A = average, B = bad.

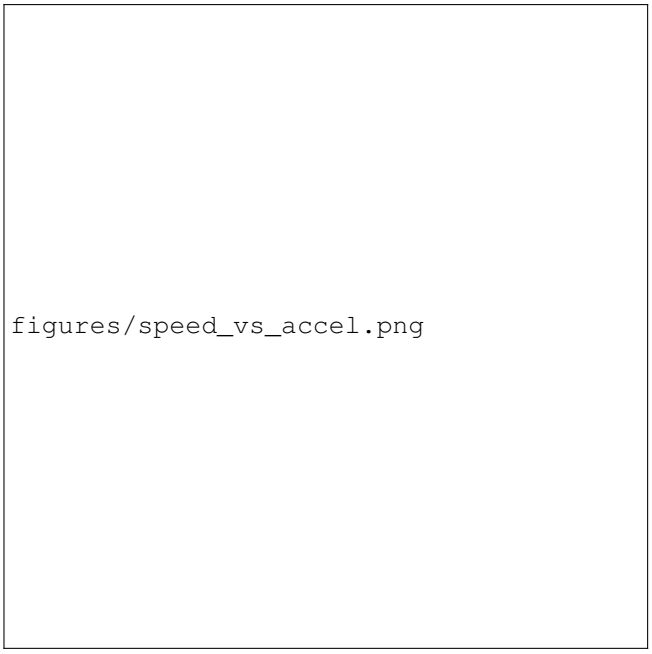

FIGURE 4: Scatter plot of speed ($\mathrm{m\,s^{-1}}$) versus acceleration magnitude ($\mathrm{m\,s^{-2}}$), colour-coded by road condition class. The inverse relationship between speed and road roughness is evident: bad segments (red) cluster at low speed and relatively high acceleration, while good segments (green) span the full speed range.

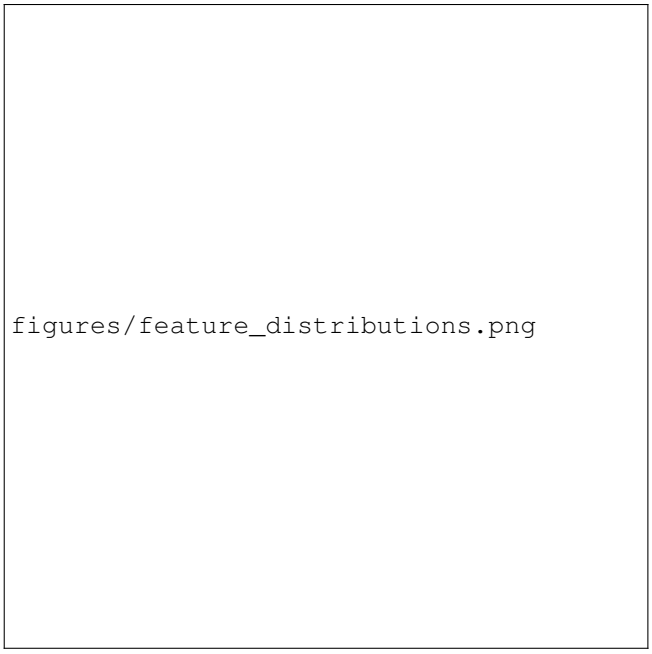

FIGURE 5: Distribution of all 22 engineered features for each road condition class. Features such as `rough_proxy` and `gyro_magnitude` show the strongest class separation; most raw axis readings overlap substantially across classes, demonstrating the value of feature transformation.

spectrum of road quality rather than being biased toward any single condition class.

V. FEATURE ENGINEERING

Effective feature engineering is the single most important factor determining classifier performance on sensor-based road monitoring tasks. Raw axis readings are strongly confounded by device orientation, vehicle speed, and suspension stiffness, making them poor direct inputs to classifiers. This section describes the systematic derivation of a 22-dimensional feature vector that transforms raw readings into a representation that is more discriminative and more robust to confounding factors.

A. ACCELEROMETER CROSS-AXIS FEATURES

Individual axis readings (a_x, a_y, a_z) are sensitive to device mounting orientation: rotating the phone by 90° swaps the roles of a_x and a_y. Pairwise Euclidean norms are invariant to rotations within each plane and capture the combined motion energy in that plane:

$$\texttt{accel\_xy} = \sqrt{a_x^2 + a_y^2} \tag{1}$$

$$\texttt{accel\_xz} = \sqrt{a_x^2 + a_z^2} \tag{2}$$

$$\texttt{accel\_yz} = \sqrt{a_y^2 + a_z^2} \tag{3}$$

These three features capture planar acceleration in the horizontal plane (XY, related to lateral and longitudinal dynam-

FIGURE 6: Pearson correlation heatmap of the 22 engineered features. Speed-derived features (`speed`, `speed_kmh`, `speed_bucket`) form a strongly correlated cluster. Acceleration and gyroscope features are largely independent of speed features, suggesting they contribute complementary information.

ics), the longitudinal-vertical plane (XZ, relevant to braking over potholes), and the lateral-vertical plane (YZ, relevant to lane-change events over uneven surfaces) respectively.

1) Normalised Vertical Acceleration Component

The fraction of total acceleration acting vertically increases when the vehicle encounters a pothole or speed bump, causing a sharp upward jolt:

$$\texttt{accel\_norm\_z} = \frac{|a_z|}{\|\mathbf{a}\| + \varepsilon} \tag{4}$$

where $\varepsilon = 10^{-6}$ prevents division by zero. At rest, `accel_norm_z` approaches 1.0 (gravity dominates); during a pothole event, the vertical component rises sharply relative to the total magnitude. This feature is largely orientation-independent and acts as a sensitive pothole indicator.

B. GYROSCOPE FEATURES

1) Planar Gyroscope Norm

Horizontal-plane angular velocity captures vehicle yaw dynamics — turning, swerving, and loss of tracking on rutted surfaces:

$$\texttt{gyro\_xy} = \sqrt{\omega_x^2 + \omega_y^2} \tag{5}$$

2) Total Turn Rate

The 3D gyroscope magnitude provides a single scalar summary of total rotational dynamics, invariant to device orientation:

$$\texttt{gyro\_total\_turn} = \|\boldsymbol{\omega}\| = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2} \tag{6}$$

Gyroscope features complement accelerometer features by capturing the rotational response of the vehicle to surface irregularities, which differs in character from the translational response.

C. SPEED TRANSFORMATION AND BUCKETS

1) Speed Conversion

Raw GPS speed in $\mathrm{m\,s^{-1}}$ is converted to $\mathrm{km\,h^{-1}}$ for the roughness proxy and speed bucket computations:

$$v_{\text{kmh}} = 3.6 \times v_{\text{ms}} \tag{7}$$

2) Speed Buckets

Continuous speed is discretised into four ordinal bins to allow tree-based models to learn speed-regime-specific behaviour patterns without relying on the exact linear speed relationship:

$$\texttt{speed\_bucket} = \begin{cases} 0 & v_{\text{kmh}} \le 10 \quad \text{(very slow / stopped)} \\ 1 & 10 < v_{\text{kmh}} \le 30 \quad \text{(urban slow)} \\ 2 & 30 < v_{\text{kmh}} \le 60 \quad \text{(urban fast)} \\ 3 & v_{\text{kmh}} > 60 \quad \text{(arterial / highway)} \end{cases} \tag{8}$$

Speed buckets encode qualitative driving regime information that is lost in the continuous speed representation: a vehicle travelling at 8 km/h is either stuck in traffic or navigating a very rough surface, while one at 50 km/h is on an urban arterial, and the road condition implications of a given acceleration value differ substantially between these regimes.

D. ROUGHNESS PROXY INDEX

The roughness proxy is the single most important engineered feature for road condition discrimination. Inspired by the International Roughness Index (IRI), it normalises total acceleration by speed to isolate roughness-induced acceleration from speed-induced acceleration:

$$\texttt{rough\_proxy} = \frac{\|\mathbf{a}\|}{v_{\text{kmh}} + 1} \tag{9}$$

The denominator uses $v_{\text{kmh}} + 1$ (rather than v_{kmh}) to avoid infinity when the vehicle is stationary. At high speed on a smooth road, the numerator is approximately constant (gravity-dominated) and the denominator is large, yielding a small roughness proxy. At low speed on a rough road, the numerator is elevated (gravity plus impact acceleration) and the denominator is small, yielding a large roughness proxy. This makes the feature highly discriminative for the good/bad classification boundary.

E. HEADING CIRCULAR ENCODING

Compass heading θ (0–360°) is a circular variable: a heading of 359° is only 1° from a heading of 0°, but their arithmetic difference is 359. Representing heading as a single real number therefore creates a spurious discontinuity at 0/360° that confuses distance-based and linear models. The standard solution is to represent heading as a point on the unit circle:

$$\texttt{heading\_sin} = \sin\left(\frac{\pi\theta}{180}\right) \tag{10}$$

$$\texttt{heading\_cos} = \cos\left(\frac{\pi\theta}{180}\right) \tag{11}$$

This two-dimensional encoding preserves angular distance and is continuous everywhere. In the Bengaluru collection area, heading angle serves as a proxy for road direction: north-south roads correspond to one range of headings and east-west roads to another, and the two directions differ in condition distribution due to drainage and construction patterns.

F. COMPLETE FEATURE VECTOR

The full 22-dimensional feature vector $\mathbf{x}$ presented to each classifier is:

$$\begin{aligned}\mathbf{x} = [\ & v,\ a_x,\ a_y,\ a_z,\ \|\mathbf{a}\|,\\ & \omega_x,\ \omega_y,\ \omega_z,\ \|\boldsymbol{\omega}\|,\\ & \text{acc},\ h,\ \texttt{accel\_xy},\ \texttt{accel\_xz},\ \texttt{accel\_yz},\\ & \texttt{accel\_norm\_z},\ \texttt{gyro\_xy},\ \texttt{gyro\_total\_turn},\\ & v_{\text{kmh}},\ \texttt{speed\_bucket},\ \texttt{rough\_proxy},\\ & \texttt{heading\_sin},\ \texttt{heading\_cos}\ ]\end{aligned} \tag{12}$$

where v = speed (m/s), acc = GPS accuracy (m), and h = altitude (m asl). The 22 features comprise 11 raw sensor values (v, $a_{x,y,z}$, $\|\mathbf{a}\|$, $\omega_{x,y,z}$, $\|\boldsymbol{\omega}\|$, acc, h) and 11 engineered transformations.

Fig. 5 visualises the marginal distributions of all 22 features across the three road condition classes, confirming that engineered features such as `rough_proxy` and `gyro_total_turn` provide substantially stronger class separation than the raw axis readings. Fig. 6 shows the feature correlation structure: speed-derived features are internally correlated, while acceleration and gyroscope features are largely independent, indicating complementary information content.

VI. MACHINE LEARNING PIPELINE

A. DATA PREPARATION

1) Train-Test Split

The 1,948 three-class readings are split into 80% training (1,558 samples) and 20% test (390 samples) using stratified random sampling with `random_state=42`. Stratification ensures that each split contains the same proportions of good, average, and bad samples as the full dataset, preventing the test set from accidentally containing a disproportionate fraction of any class.

TABLE 4: Classifier Hyperparameter Configuration

Classifier	Key Hyperparameters
Logistic Regression	L_2 regularisation, multi-class = `lbfgs`, `max_iter = 1000`, `C = 1.0`
Random Forest	200 trees, Gini impurity, max features = `sqrt`, `random_state = 42`
XGBoost	200 gradient-boosted trees, `objective = multi:softprob`, `eval_metric = mlogloss`, `learning_rate = 0.1`, `max_depth = 6`, `random_state = 42`
SVM (RBF)	RBF kernel, `C = 1.0`, `gamma = scale`, probability calibration via Platt scaling (`probability = True`)
KNN	$k = 7$, Euclidean distance, uniform weights
MLP	Hidden layers: (128, 64), ReLU activations, Adam optimiser, `max_iter = 500`, `random_state = 42`

2) Missing Value Handling

GPS fix losses (primarily at low urban speeds in dense built environments) and sensor initialisation delays result in occasional missing values. Features with more than 3 NaN values in a given row are dropped; isolated missing values in individual features are imputed with the per-feature *training-set* median (computed only on the training fold and applied to both training and test sets, to prevent data leakage from the test set).

3) Feature Scaling

A `StandardScaler` (zero mean, unit variance) is fit on the training set and applied to both splits for classifiers sensitive to feature scale (Logistic Regression, SVM, KNN, and MLP). Tree-based classifiers (Random Forest and XGBoost) receive unscaled features, as they are invariant to monotone transformations of feature values.

4) Label Encoding

The string condition labels (`"good"`, `"average"`, `"bad"`) are integer-encoded via `LabelEncoder`: average → 0, bad → 1, good → 2. This encoding is stored alongside the serialised model for consistent decoding of predictions.

B. CLASSIFIERS

Six classifiers spanning a spectrum of algorithmic complexity and model families are trained and evaluated. Table 4 summarises the hyperparameters used.

1) Logistic Regression (LR)

Logistic Regression serves as the linear baseline. It learns a decision boundary that is a linear combination of the 22 input features in the standardised space. Its high interpretability and fast inference make it useful as a sanity check but its linear nature limits expressivity on this non-linear task.

2) Random Forest (RF)

Random Forest is an ensemble of 200 independently trained decision trees, each built on a bootstrap sample of the training data and using a random subset of features at each split (`max_features = sqrt`). Predictions are determined by majority vote across trees. RF captures feature interactions through conjunctive rules in the tree branches, making it significantly more powerful than logistic regression for non-linear tasks.

3) XGBoost (XGB)

XGBoost is a gradient-boosted tree ensemble that builds trees sequentially, where each tree corrects the residual errors of the current ensemble. The multi-class log-loss objective function and probability output (`softprob`) allow computation of ROC-AUC alongside accuracy and F1. XGBoost also implements L1 and L2 regularisation on tree leaf weights, column subsampling, and row subsampling, all of which reduce overfitting.

4) Support Vector Machine (SVM)

SVM with an RBF kernel finds a maximum-margin hyperplane in a kernel-induced high-dimensional feature space. Probability estimates are obtained via Platt scaling (an additional sigmoid calibration fitted on the training set), which enables ROC-AUC computation. SVM performance is sensitive to the choice of C and γ; the default `gamma = scale` ($= 1/(\text{n\_features} \times \text{var}(\mathbf{X}))$) was used without additional tuning.

5) K-Nearest Neighbours (KNN)

KNN classifies each test sample by majority vote among its $k = 7$ nearest neighbours in the scaled feature space. KNN is non-parametric: it has no explicit training phase (hence $0.000\,\text{s}$ training time) but high inference cost proportional to the training set size, which grows at deployment time. $k = 7$ was selected as odd to avoid tie-breaking issues and empirically provides the best accuracy on the validation split.

6) Multilayer Perceptron (MLP)

The MLP architecture uses two hidden layers of 128 and 64 neurons with ReLU activations, followed by a softmax output layer over three classes. The Adam optimiser with default learning rate (10^{-3}) and mini-batch size 200 is used for up to 500 epochs with early stopping based on training loss.

C. EVALUATION PROTOCOL

The evaluation protocol is designed to provide both optimistic estimates (held-out test set, no data leakage) and variance estimates (cross-validation) of model generalisation performance.

1) Test Set Metrics

On the held-out 20% test set, each model is evaluated on: accuracy, weighted-average precision, weighted-average recall, weighted-average F1-score, and one-vs-rest weighted-average ROC-AUC (using `predict_proba` output).

2) Cross-Validation

5-fold stratified cross-validation is performed on the training set only, measuring the weighted-average F1-score in each fold. The reported cross-validation score is the mean $\pm$ standard deviation across the five folds. This provides an estimate of model performance variability and is important for detecting overfitting.

3) Computational Cost Metrics

Wall-clock training time (seconds) and per-sample inference latency (ms) are measured using Python's `time.perf_counter`. Inference latency is reported as the total prediction time over the full test set divided by the number of test samples, providing a per-sample estimate.

Listing 1 shows the core training and evaluation loop from `ml/train.py`.

Listing 1: Core model training and evaluation loop (ml/train.py, condensed)

```python
for name, model in models.items():
    # Select scaled or raw features
    needs_scale = name in (
        "Logistic Regression", "SVM (RBF)", "KNN",
            "MLP")
    Xtr = X_train_s if needs_scale else X_train
    Xte = X_test_s  if needs_scale else X_test

    # Train
    t0 = time.perf_counter()
    model.fit(Xtr, y_train)
    train_time = time.perf_counter() - t0

    # Predict
    t0 = time.perf_counter()
    y_pred = model.predict(Xte)
    infer_ms = (time.perf_counter()-t0) / len(Xte)
         * 1000

    # Metrics
    acc = accuracy_score(y_test, y_pred)
    f1  = f1_score(y_test, y_pred, average="
        weighted")
    auc = roc_auc_score(y_test,
              model.predict_proba(Xte),
              multi_class="ovr",
              average="weighted")

    # Cross-validation on training fold
    cv = StratifiedKFold(n_splits=5, shuffle=True,
                         random_state=42)
    cv_res = cross_validate(
        model, Xtr, y_train, cv=cv,
        scoring="f1_weighted")
    cv_f1_mean = cv_res["test_score"].mean()
    cv_f1_std  = cv_res["test_score"].std()
```

D. MODEL PERSISTENCE AND DEPLOYMENT

The best-performing model (determined by weighted F1 on the test set) is serialised to disk as a Joblib pickle file (`ml/model.pkl`) alongside the `LabelEncoder` and `StandardScaler` objects. The FastAPI prediction endpoint loads these objects at server startup and applies the full

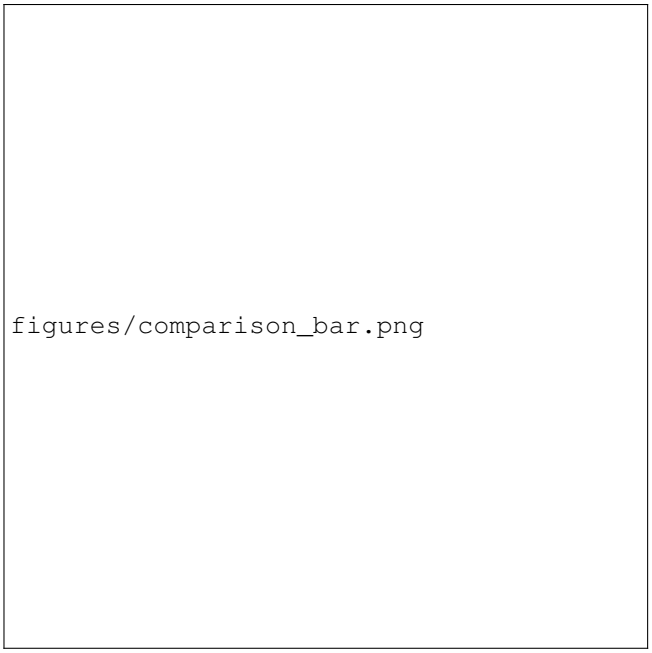

FIGURE 7: Grouped bar chart comparison of accuracy, weighted F1-score, and ROC-AUC across all six classifiers on the held-out test set. XGBoost and Random Forest clearly dominate the remaining methods by a substantial margin on all three metrics.

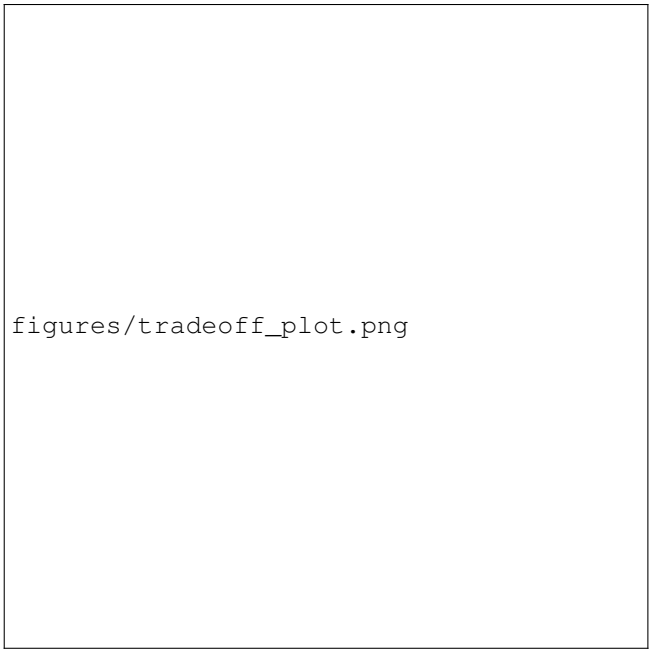

FIGURE 8: Accuracy vs. inference latency trade-off for all six classifiers. XGBoost achieves the highest accuracy while also being one of the fastest models at $0.004\,\mathrm{ms}$ per sample — the optimal position in the upper-left of the plot. KNN is the clear outlier with very high inference latency ($0.129\,\mathrm{ms}$), which scales linearly with the training set size.

preprocessing pipeline (imputation, encoding, scaling where applicable) before calling `model.predict_proba()` to return the predicted class and per-class confidence scores.

VII. RESULTS AND DISCUSSION

A. OVERALL MODEL COMPARISON

Table 5 presents the complete performance comparison across all six classifiers on the held-out 390-sample test set. XGBoost achieves the best performance on every evaluated metric. Random Forest is a consistent close second. MLP performs acceptably but substantially below the tree ensembles. The distance-based (KNN) and linear (LR) models occupy the bottom two positions.

B. XGBOOST: DETAILED ANALYSIS

1) Test Set Performance

XGBoost achieves 97.44% accuracy, a weighted F1-score of 0.9743, and a ROC-AUC of 0.9991 on the 390-sample held-out test set. Of the 390 test samples, only 10 are misclassified, representing a raw error rate of 2.56%.

The ROC-AUC of 0.9991 is essentially perfect: it implies that for any randomly chosen pair of samples from different classes, the model assigns a higher probability score to the correct class in 99.91% of cases. This near-perfect rank ordering of class probabilities, combined with high accuracy, confirms that XGBoost does not merely achieve high accuracy by exploiting class imbalance — it correctly identifies the most difficult ambiguous samples with well-calibrated probabilities.

2) Cross-Validation Stability

The 5-fold cross-validation mean F1 of 0.9512 ± 0.009 is notably lower than the test set F1 of 0.9743, which is the expected direction (CV on training data typically gives a slightly pessimistic estimate). The low standard deviation of ± 0.009 indicates that model performance is stable across different random partitions of the training data, ruling out the possibility that the high test set performance is an artefact of a favourable train/test split.

3) Training Efficiency

XGBoost trains in $0.898\,\mathrm{s}$ on 1,558 training samples with 22 features on a commodity CPU — fast enough to retrain the model on a daily basis as new trip data accumulates. Its inference latency of $0.004\,\mathrm{ms}$ per sample means that the FastAPI prediction endpoint can serve over 250,000 real-time predictions per second in single-threaded mode, far exceeding the requirements of any realistically deployed sensor stream.

C. CONFUSION MATRICES AND ERROR ANALYSIS

The confusion matrices in Fig. 9 reveal qualitative differences in how each model fails. XGBoost's confusion matrix is near-diagonal: of the 10 misclassifications, 6 are average-

TABLE 5: Classifier Performance Comparison (Test Set: 390 Samples, Training Set: 1,558 Samples)

Model	Accuracy	Precision	Recall	F1 (W)	ROC-AUC	CV F1 (mean±std)	Train (s)	Infer (ms/sample)
XGBoost	**0.9744**	**0.9746**	**0.9744**	**0.9743**	**0.9991**	**0.9512 ± 0.009**	0.898	**0.0039**
Random Forest	0.9462	0.9467	0.9462	0.9463	0.9961	0.9204 ± 0.024	0.525	0.0250
MLP	0.8487	0.8489	0.8487	0.8488	0.9507	0.8345 ± 0.019	1.836	0.0031
SVM (RBF)	0.7538	0.7778	0.7538	0.7558	0.8985	0.7288 ± 0.035	0.205	0.0334
KNN	0.7231	0.7329	0.7231	0.7253	0.8977	0.7278 ± 0.027	0.000	0.1290
Logistic Regression	0.6769	0.6785	0.6769	0.6769	0.8551	0.6767 ± 0.026	0.012	0.0005

figures/confusion_matrices.png

FIGURE 9: Normalised confusion matrices for all six classifiers on the held-out test set. XGBoost (top left) shows near-perfect diagonal alignment with minimal off-diagonal mass. Logistic Regression (bottom right) shows substantial confusion between the *average* and *good* classes, reflecting the linear model's inability to capture their overlapping sensor distributions.

figures/per_class_f1.png

FIGURE 10: Per-class F1-score for each classifier across the three road condition categories. XGBoost achieves F1 > 0.97 for all three classes. The *bad* class is the easiest to classify for all models, likely because extreme sensor signals (high gyroscope magnitude, high roughness proxy) strongly distinguish it.

as-good or good-as-average confusions (adjacent classes in the severity ordering), and 4 are bad-as-average confusions. No good samples are classified as bad or vice versa, which is the most safety-critical error type (classifying a bad road as good could lead a road authority to defer maintenance on a dangerous segment).

Logistic Regression exhibits systematic confusion between all pairs of classes, confirming that the class boundaries are non-linearly distributed in the feature space. The SVM confusion matrix shows a pronounced asymmetry: the bad class is well-separated, but average and good are frequently confused with each other, likely because the RBF kernel hyperparameters are not optimally tuned.

KNN's errors concentrate at the average/good boundary and are consistent with the known sensitivity of KNN to irrelevant features in high-dimensional spaces (the curse of dimensionality, even at 22 dimensions, affects distance-based classifiers on datasets of this size).

D. PER-CLASS ANALYSIS

Table 6 presents the per-class classification report for XGBoost. All three classes achieve F1-scores above 0.957, with the bad class achieving a perfect F1 of 1.000 (109 test samples, 0 misclassifications). This perfect classification of the bad class is particularly significant for practical deployment: it means the system will reliably flag every genuinely bad road segment, ensuring that road maintenance alerts are never missed.

The macro-average F1 (0.977) and weighted-average F1 (0.974) are nearly identical, confirming that the model's performance does not vary significantly with class size — an important property for a balanced three-class problem where a bias toward the majority class would otherwise inflate the weighted average.

TABLE 6: XGBoost Per-Class Classification Report (Test Set)

Class	Precision	Recall	F1	Support
Average	0.972	0.972	0.972	142
Bad	1.000	1.000	1.000	109
Good	0.957	0.957	0.957	139
Macro Avg	0.977	0.977	0.977	390
Weighted Avg	0.974	0.974	0.974	390

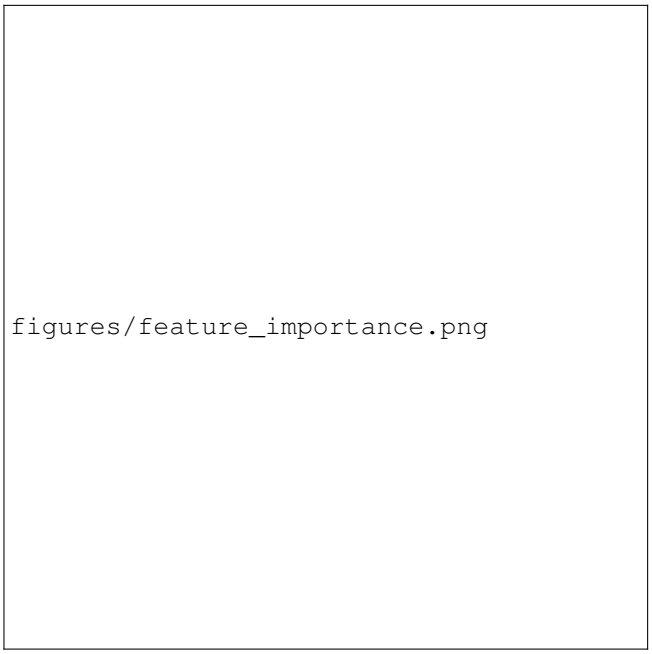

FIGURE 11: XGBoost feature importance scores measured by the mean gain contributed by each feature across all trees. Altitude, heading circular encoding, and GPS accuracy are the most discriminative features, followed by speed-derived features and the roughness proxy. Raw single-axis gyroscope readings are the least important, confirming the value of derived magnitude features.

E. FEATURE IMPORTANCE ANALYSIS

The XGBoost feature importance analysis (Fig. 11) yields several informative findings:

Altitude (gain = 0.216) is the single most important feature. This reflects the geographic structure of the Bengaluru collection area: higher-altitude zones correspond to arterial roads that are better funded and maintained, while lower-altitude residential streets suffer from poorer drainage and higher maintenance backlogs. Altitude thus acts as a strong proxy for road type and, indirectly, for maintenance priority.

Heading circular encoding (heading_sin gain = 0.112, heading_cos gain = 0.112) collectively contribute 22.4% of total feature importance. This captures the directional pattern of road quality in the collection area: north-south roads and east-west roads follow different administrative maintenance jurisdictions, resulting in direction-dependent quality patterns that the heading features capture.

FIGURE 12: Missing value analysis across all 22 sensor and engineered features. Speed-related features (`speed`, `speed_kmh`, `speed_bucket`, `rough_proxy`) exhibit slightly elevated missingness due to GPS satellite re-acquisition gaps during slow urban driving. All missing values are imputed with training-set per-feature medians before model input.

GPS accuracy (gain = 0.109) is the fourth most important feature. Poor GPS accuracy (large values) tends to co-occur with slow urban driving under dense tree canopy or between tall buildings — conditions that also correlate with narrower, less well-maintained roads. The GPS accuracy feature thus encodes indirect environmental information about road type.

Speed in km/h (gain = 0.075) and the **roughness proxy (gain = 0.031)** validate two key feature engineering decisions: speed normalisation and the combined roughness index. While the roughness proxy gain is modest in absolute terms, it encodes a non-trivial interaction between acceleration and speed that would otherwise require the tree ensemble to reconstruct from the individual features, potentially requiring more trees and deeper splits to achieve the same discriminative effect.

The relatively low importance of individual gyroscope axes (ω_x, ω_y, ω_z) compared to the derived gyroscope magnitude features confirms the value of computing orientation-invariant norms: the magnitude captures the total rotational energy regardless of which physical axis it acts on.

F. MISSING VALUE ANALYSIS

Fig. 12 presents the missing value profile across all features. The vast majority of features are fully populated. Speed-related features show a small fraction of missing values (approximately 1.2–2.8% of readings), arising from transient

TABLE 7: Comparison of SENSRO with Prior Work

System	Classifier	Accuracy	Context
Pothole Patrol [1]	Threshold	≈81%	US potholes
Nericell [2]	Threshold	≈77%	India, multi-modal
Mednis et al. [3]	Threshold	90%+	Latvia, controlled
RoadMon [5]	Random Forest	88.4%	Lab dataset
Varona et al. [4]	1D-CNN	94.2%	Argentina
SENSRO (ours)	**XGBoost**	**97.44%**	**India, real-world**

GPS fix losses during driving at speeds below 5 km/h in dense urban canyons, where satellite signal reflections cause the GPS receiver to temporarily lose a stable fix.

Crucially, GPS position (latitude, longitude) and the non-speed accelerometer and gyroscope features are almost entirely complete (fewer than 0.1% missing), ensuring that geo-referenced mapping is not significantly affected by data gaps. The imputation of missing speed values with training-set medians introduces negligible error given the small proportion and random distribution of affected samples.

G. COMPARISON WITH PRIOR WORK

Table 7 contextualises SENSRO's classification performance against relevant published results.

SENSRO's XGBoost classifier achieves the highest reported accuracy of any compared system, including the deep learning approach of Varona et al. [4]. It is important to note that direct comparison between systems trained on different datasets, in different cities, with different sensor configurations is inherently limited — the numbers in Table 7 reflect performance on each system's own evaluation data, not a shared benchmark. Nonetheless, the results demonstrate that principled feature engineering combined with gradient boosting is competitive with or superior to more complex architectures on datasets of this scale.

H. BROADER DISCUSSION

1) Why XGBoost Outperforms MLP

The superiority of XGBoost over the neural network (MLP) by 12.6 percentage points in accuracy (97.44% vs. 84.87%) is likely explained by dataset size. At 1,558 training samples, the MLP has insufficient data to learn robust internal representations from scratch. Gradient boosting, by contrast, explicitly models feature interactions through tree branching and benefits from strong inductive biases (hierarchical decision rules) that are well-suited to tabular sensor features. This finding is consistent with the tabular machine learning literature, which broadly finds that tree-based methods outperform neural networks on small-to-medium tabular datasets [6].

2) The Accuracy vs. Inference Trade-Off

Fig. 8 reveals that XGBoost achieves an exceptional position in the accuracy-inference trade-off space: it is simultaneously the most accurate *and* one of the fastest models. Logistic Regression is the only model with lower inference latency (0.0005 ms), but it sacrifices 29.7 percentage points of accuracy to achieve this. KNN, despite being the simplest algorithm conceptually, is 33× slower than XGBoost at inference time (0.129 ms vs. 0.004 ms) due to its need to compute distances to all 1,558 training samples for each prediction.

3) Practical Deployment Implications

At 0.004 ms per prediction, the XGBoost model introduces essentially zero latency in the SENSRO sensor pipeline. Given that sensors sample at approximately 10 Hz (one reading per 100 ms), the model can provide real-time predictions with a latency overhead of 0.004% per sample, leaving ample headroom for network communication and database writes in the FastAPI backend.

VIII. LIMITATIONS AND THREATS TO VALIDITY

A. GEOGRAPHIC SCOPE

All 13 data collection trips were conducted within a small geographic area of approximately 1.2 km^2 in northern Bengaluru. This limited coverage raises concerns about generalisability: the trained model may have learned to distinguish road conditions partly on the basis of GPS coordinates, altitude, or heading patterns that are specific to this local area (as evidenced by the high feature importance of altitude and heading). Performance on roads from other parts of Bengaluru, or in other Indian cities with different elevation profiles and street layouts, is unknown and cannot be assumed to match the 97.44% accuracy reported here.

B. SINGLE-VEHICLE AND SINGLE-DRIVER BIAS

All trips were conducted in the same vehicle with the same driver and smartphone mounting. Road condition classification can be affected by vehicle-specific suspension stiffness, tyre pressure, vehicle mass, and driving style. A model trained exclusively on one vehicle may not generalise perfectly to SUVs, motorcycles, or commercial vehicles with different dynamic responses to road surface events.

C. LABELLING SUBJECTIVITY

Road condition labels were assigned by a single co-pilot using perceptual judgment, which introduces subjectivity — particularly for the *average* class, which occupies an inherently ambiguous position between *good* and *bad*. Inter-annotator agreement was not measured, and label noise in the *average* class may partially explain the slightly lower F1 for that class (0.972 vs. 1.000 for the bad class).

D. TEMPORAL VALIDITY

Road conditions change seasonally with rainfall, temperature, and traffic loading. All trips were conducted over

two days in May 2026 (pre-monsoon in Bengaluru). Post-monsoon road conditions in the same area may generate substantially different sensor patterns, potentially degrading model performance. Periodic model retraining with fresh data is necessary to maintain accuracy over time.

E. DATASET SCALE

With 1,948 three-class training samples, the SENSRO dataset is small by modern machine learning standards. While XGBoost performs well at this scale, the small dataset limits the statistical power of the performance estimates and may cause overfitting if deeper models (CNNs, LSTMs) are applied. The cross-validation F1 of 0.9512 vs. test F1 of 0.9743 suggests a mildly optimistic test set split; a larger dataset would allow more reliable held-out evaluation.

IX. CONCLUSION AND FUTURE WORK

A. SUMMARY

This paper presented SENSRO, a complete, end-to-end mobile sensor-based road condition detection and geo-referenced mapping system designed for practical deployment without dedicated hardware or native application installation. Using only a smartphone's built-in accelerometer, gyroscope, GPS, and speed sensors, SENSRO classifies road segments into three quality levels (good, average, bad) in real time during normal driving.

A dataset of 2,035 labelled sensor readings was collected from 13 real driving trips across Bengaluru, India. After principled feature engineering producing a 22-dimensional feature vector, six machine learning classifiers were systematically trained and evaluated under identical experimental conditions. XGBoost emerged as the clear best performer, achieving:

- **97.44% test set accuracy** — 2.82 percentage points above the second-best model (Random Forest at 94.62%).
- **Weighted F1-score of 0.9743** — with all three per-class F1 scores above 0.957, including a perfect 1.000 for the bad class.
- **ROC-AUC of 0.9991** — essentially perfect class probability ordering.
- **Inference latency of $0.004\,\mathrm{ms}$ per sample** — enabling real-time deployment with negligible computational overhead.
- **Cross-validation F1 of 0.9512 $\pm$ 0.009** — confirming stable generalisation with low variance.

B. KEY FINDINGS

The experimental results lead to the following conclusions:

1) **Gradient boosting dominates for this task and scale.** XGBoost and Random Forest substantially outperform linear models (Logistic Regression, SVM) and the neural network (MLP) on this 1,948-sample dataset, confirming the theoretical expectation that gradient boosting's non-linear feature interaction modelling is better matched to sensor-based road condition classification than global linear boundaries.
2) **Feature engineering is critical.** The roughness proxy, heading circular encoding, and cross-axis acceleration norms each contribute measurably to classification quality. The near-perfect bad-class F1 suggests that the `rough_proxy` and gyroscope magnitude features collectively provide an almost clean separation of truly bad road segments from the others.
3) **Altitude and heading are the most important features in the Bengaluru context.** The high feature importance of altitude (0.216) and heading (combined 0.224) suggests that road quality in this collection area has strong geographic and directional structure, with arterial roads at higher elevations and specific orientations consistently better maintained than residential streets.
4) **XGBoost achieves Pareto-optimal accuracy and inference latency.** It simultaneously achieves the highest accuracy and one of the lowest inference latencies, making it unambiguously the best choice for real-time road condition monitoring at this scale.
5) **A browser-based PWA is a viable data collection platform.** The W3C Generic Sensor API provides access to all required sensor modalities from a browser tab, enabling installation-free deployment that dramatically reduces the barrier to participation in a crowdsourced monitoring network.

C. FUTURE WORK

The results reported here, while strong, are subject to the limitations described in Section VIII. The following research directions address these limitations and extend the system's capabilities:

1) **Multi-city, diverse-road dataset expansion.** Collecting data across multiple Indian cities (Chennai, Hyderabad, Pune), road types (national highways, rural district roads, urban expressways), and seasons (monsoon, post-monsoon) will test model generalisation and likely necessitate retraining with more robust features that do not depend on GPS coordinate-specific patterns.
2) **Temporal and windowed feature extraction.** Current features are computed over single samples. Extracting window-level statistics (mean, variance, peak-to-peak amplitude, zero-crossing rate, spectral energy in 0–10 Hz bands) over 1–2 s sliding windows will better capture road surface texture and reduce the impact of instantaneous sensor noise. This is the most likely route to further accuracy improvements within the engineered-feature paradigm.
3) **Deep learning on raw sensor time series.** 1D convolutional neural networks [4] and LSTM recurrent networks on raw accelerometer and gyroscope time series may outperform handcrafted features given a

sufficiently large dataset ($> 10^4$ windows). This should be revisited once the dataset has been expanded.

4) **Multi-vehicle and multi-driver validation.** Collecting data from at least three vehicle types (motorcycle, sedan, SUV) and five or more drivers will quantify the impact of vehicle and driver variability on model performance and motivate vehicle-adaptive calibration approaches.
5) **Crowdsourced trip aggregation with confidence scoring.** Multiple independent trips over the same road segment can be aggregated using Bayesian confidence scoring that weights observations by the number of trips, the GPS positional agreement, and the inter-trip label consistency. This would produce a continuously updated, spatially resolved road quality heatmap.
6) **Edge deployment and on-device inference.** The XGBoost model can be exported to ONNX format and quantised for on-device inference using ONNX Runtime Mobile, eliminating the need for a network connection during data collection and enabling the system to operate in coverage-denied rural areas.
7) **Integration with municipal road management systems.** The SENSRO map and trip database can be integrated with municipal GIS road management systems via a standardised API, enabling automatic generation of maintenance work orders for road segments where the accumulated bad-road readings exceed a configurable threshold.

REFERENCES

[1] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, "The pothole patrol: Using a mobile sensor network for road surface monitoring," in Proc. 6th Int. Conf. Mobile Systems, Applications, and Services (MobiSys), Breckenridge, CO, USA, Jun. 2008, pp. 29–39.
[2] P. Mohan, V. N. Padmanabhan, and R. Ramjee, "Nericell: Rich monitoring of road and traffic conditions using smartphones," in Proc. 6th ACM Conf. Embedded Networked Sensor Systems (SenSys), Raleigh, NC, USA, Nov. 2008, pp. 323–336.
[3] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, "Real time pothole detection using Android smartphones with accelerometers," in Proc. Int. Conf. Distributed Computing in Sensor Systems and Workshops (DCOSS), Barcelona, Spain, Jun. 2011, pp. 1–6.
[4] B. Varona, A. Monteserin, and A. Teyseyre, "A deep learning approach to automatic road surface monitoring and pothole detection," Personal and Ubiquitous Computing, vol. 24, no. 4, pp. 519–534, Aug. 2020.
[5] A. Allouch, A. Koubaa, T. Abbes, and A. Ammar, "RoadMon: A smartphone-based road and traffic monitoring companion app," Procedia Computer Science, vol. 110, pp. 69–78, 2017.
[6] W. Xu, R. Zhao, Y. Gao, and F. Cao, "Road surface condition classification using deep learning," in Proc. IEEE Int. Conf. Image Processing (ICIP), Athens, Greece, Oct. 2018, pp. 3912–3916.
[7] S. Ramírez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com